

Chapter 3:

Local Search and Optimization Problems

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Optimization Problems
- Local Search Strategies
- State Space Landscape
- Hill Climbing Search
- Variants of Hill Climbing Search
- Simulated Annealing
- Local Beam Search
- Evolutionary Algorithms

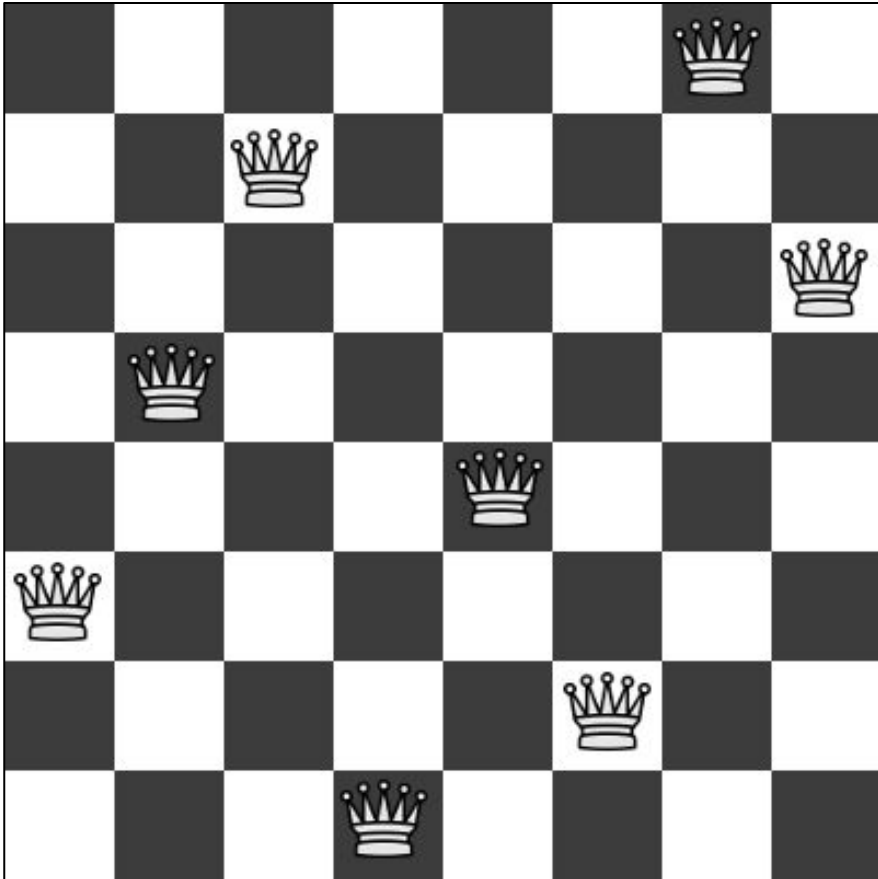
Classical Search Strategies

- Search strategy: Classical search algorithms
 - Explore search spaces systematically.
 - Keep track of expanded and unexpanded nodes.
- Solution
 - A fixed sequence of actions.
- Issue
 - The path to the goal is irrelevant in many problems.

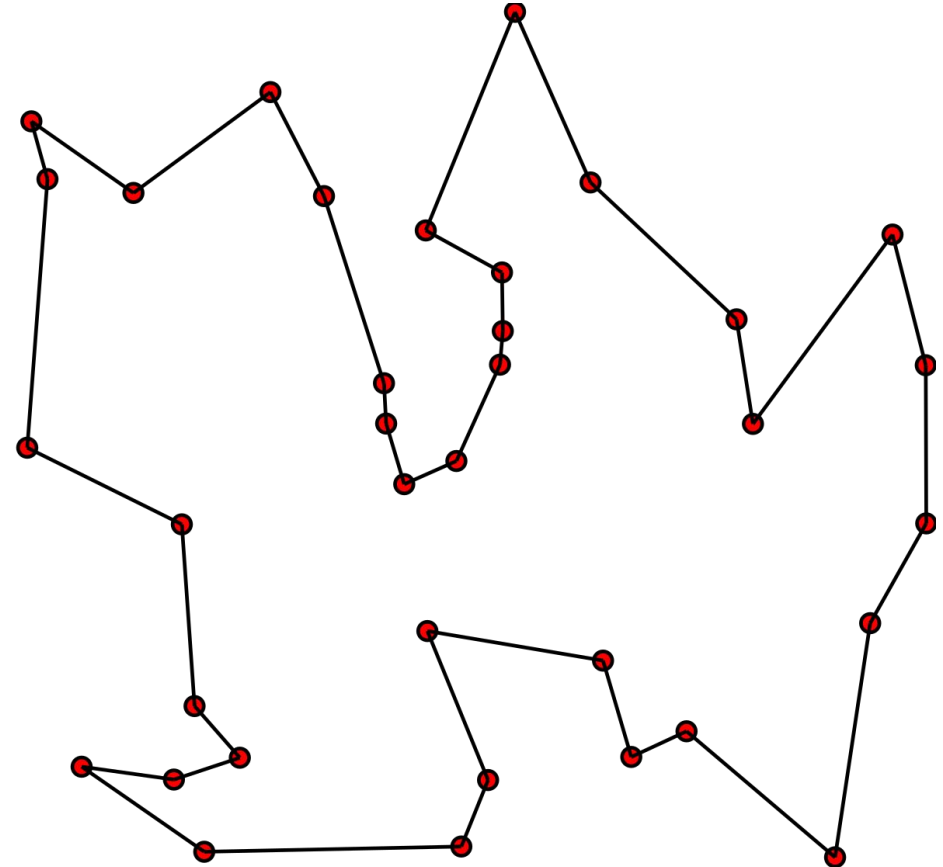
Optimization Problems

- An **optimization problem** is the problem of finding the best solution from all feasible solutions.
- Algorithms for solving optimization problems narrow the set of candidate solutions down to a subset of the feasible solutions, according to an **objective function**.
- **Candidate solutions** refer to a set of possible solutions of a given problem.

Optimization Problems



The 8-Queen Problem

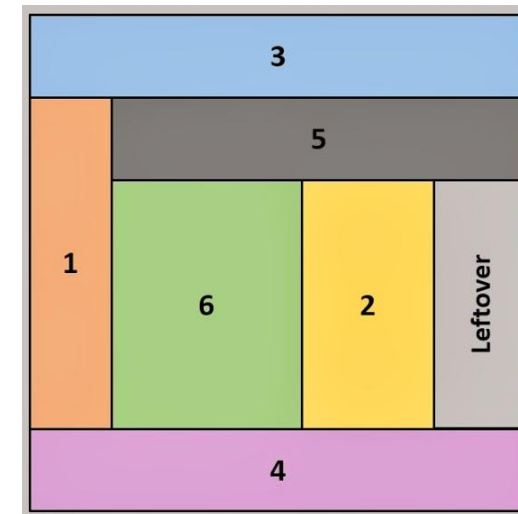


Traveling-Salesman Problem

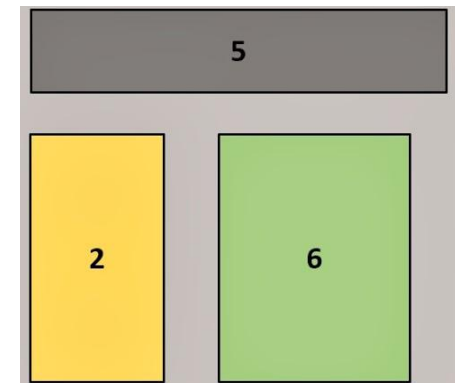
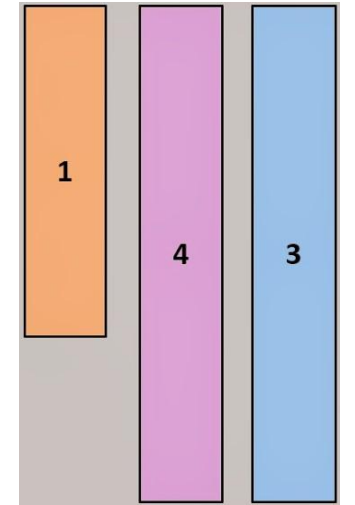
Optimization Problems

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
9:00 AM		9:00 AM 11:30 AM			9:00 AM 10:15 AM	
9:15 AM	9:15 AM 11:00 AM	World War II		9:15 AM 11:15 AM	Chemical Reactions	
9:30 AM	Students will be introduced to the basics like variables, substitution, evaluation of expressions.	This course will cover all aspects of WWII in detail like the pre-war events, course of the war and its impact on the modern world.		This course will teach you the basics of kinematics like velocity, speed and acceleration.	We will cover topics like chemical equations, chemical equilibrium, reaction types and catalysts.	
9:45 AM						
10:00 AM			10:00 AM 12:15 PM			10:00 AM 11:30 AM
10:15 AM	Algebra 101		Students will be introduced to the basics like variables, substitution, evaluation of expressions.	Kinematics		This course will teach you the basics of kinematics like velocity, speed and acceleration.
10:30 AM						
10:45 AM					10:45 AM 1:15 PM	
11:00 AM					World War II	
11:15 AM	11:15 AM 1:45 PM				This course will cover all aspects of WWII in detail like the pre-war events, course of the war and its impact on the modern world.	
11:30 AM	This course will cover all aspects of WWII in detail like the pre-war events, course of the war and its impact on the modern world.	11:45 AM 1:15 PM				
11:45 AM		Kinematics				
12:00 PM		This course will teach you the basics of kinematics like velocity, speed and acceleration.			12:00 PM 1:15 PM	
12:15 PM					We will cover topics like chemical equations, chemical equilibrium, reaction types and catalysts.	
12:30 PM			12:30 PM 1:45 PM	12:30 PM 2:15 PM		
12:45 PM	World War II		Chemical Reactions	Algebra 101		
1:00 PM			We will cover topics like chemical equations, chemical equilibrium, reaction types and catalysts.	Students will be introduced to the basics like variables, substitution, evaluation of expressions.		
1:15 PM						
1:30 PM						
1:45 PM						
2:00 PM						

Course Scheduling Problem



Cutting Stock Problem



Local Search Strategies

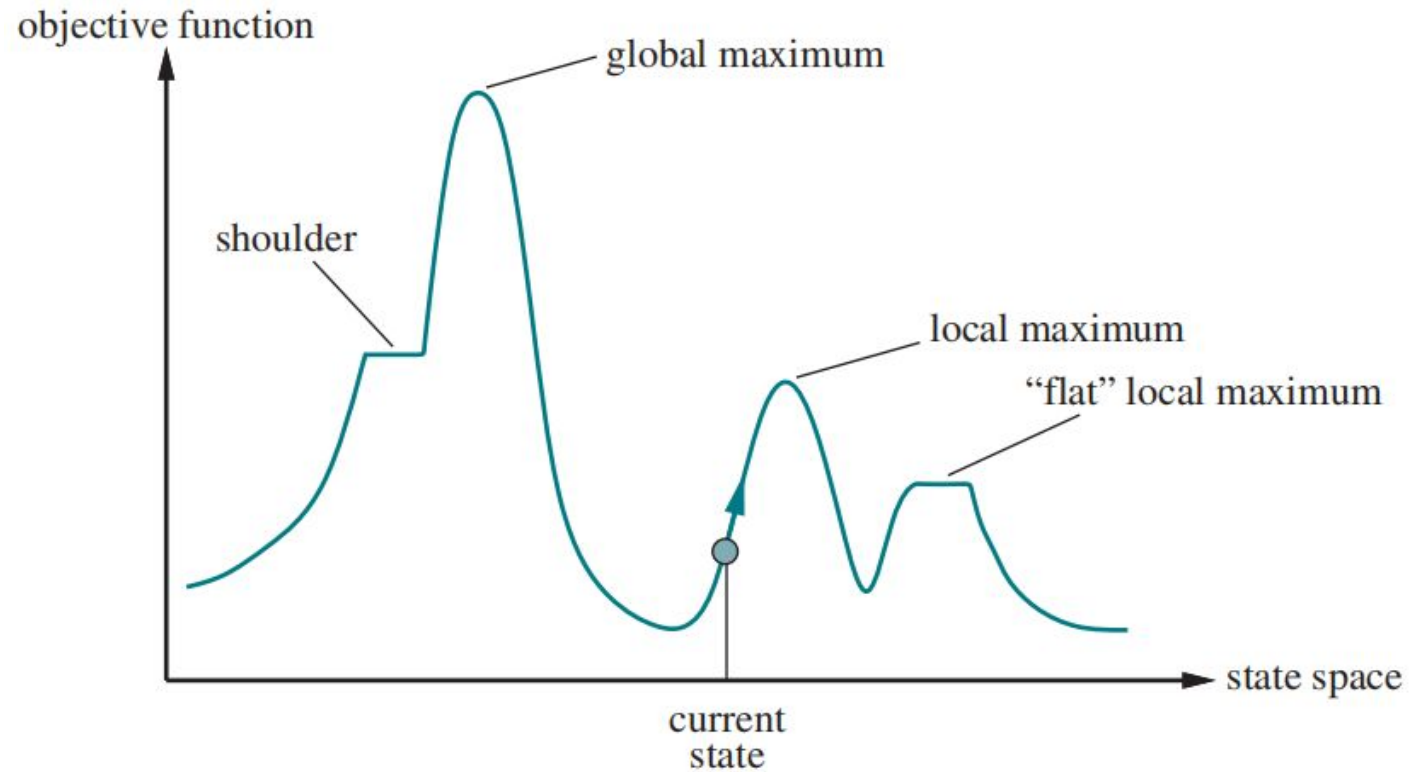
- Search problems
 - Optimization problems
- Solution
 - The goal state itself
- Search strategy
 - Local search algorithms

Local Search Strategies

- **Local search algorithms** operate by searching from a start state to neighboring states:
 - The decision is made solely based on the information about the neighbourhood of the current state.
 - The paths followed by the search or reached states are not retained.
 - The search is not systematic.
- **Strengths**
 - Have very low memory consumption.
 - Often can find reasonable solutions in large or infinite state spaces.

State Space Landscape

- A **state-space landscape** is a **graphical representation** of different states of a problem reachable by a search algorithm.



State Space Landscape

□ **Current state**

- The state an agent is currently present.

□ **Global maximum**

- The best possible state with the highest value of objective function.

□ **Local maximum**

- A state which is better than its neighbor states, but not the best one.

□ **Flat local maximum**

- All neighboring states of the current state have the same value.

□ **Shoulder**

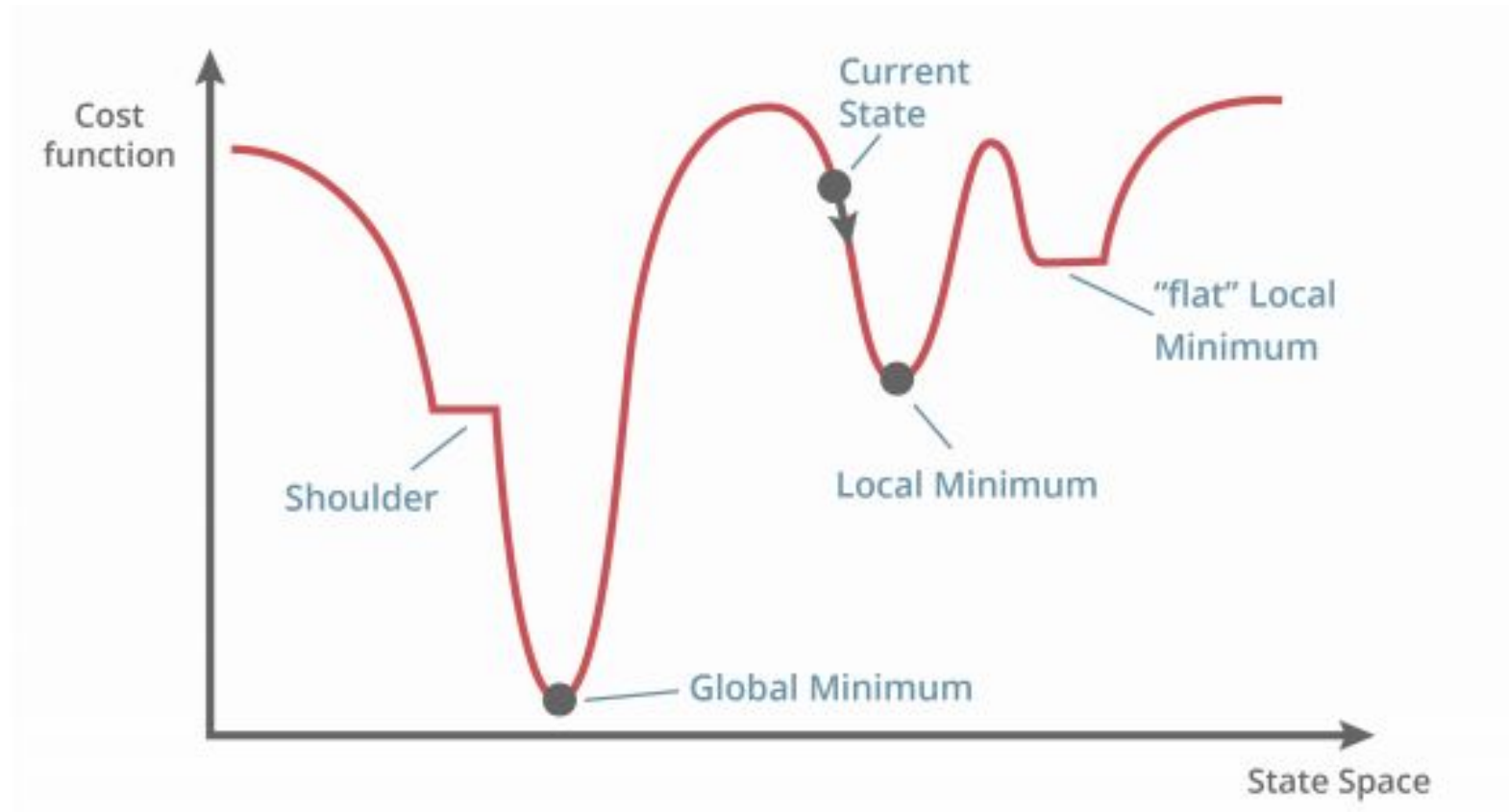
- A plateau whose edge is stretching upwards.

State Space Landscape

- Each state (point) in the landscape has an "**elevation**", defined by the value of an objective function.
- If the elevation corresponds to:
 - **Objective function**
 - The aim is to find the **global maximum**.
 - The process **hill climbing**.
 - **Heuristic cost function**
 - The aim is to find the **global minimum**.
 - It is referred to as **gradient descent**.

State Space Landscape

- An example of state space landscape when the elevation corresponds to a heuristic cost function.

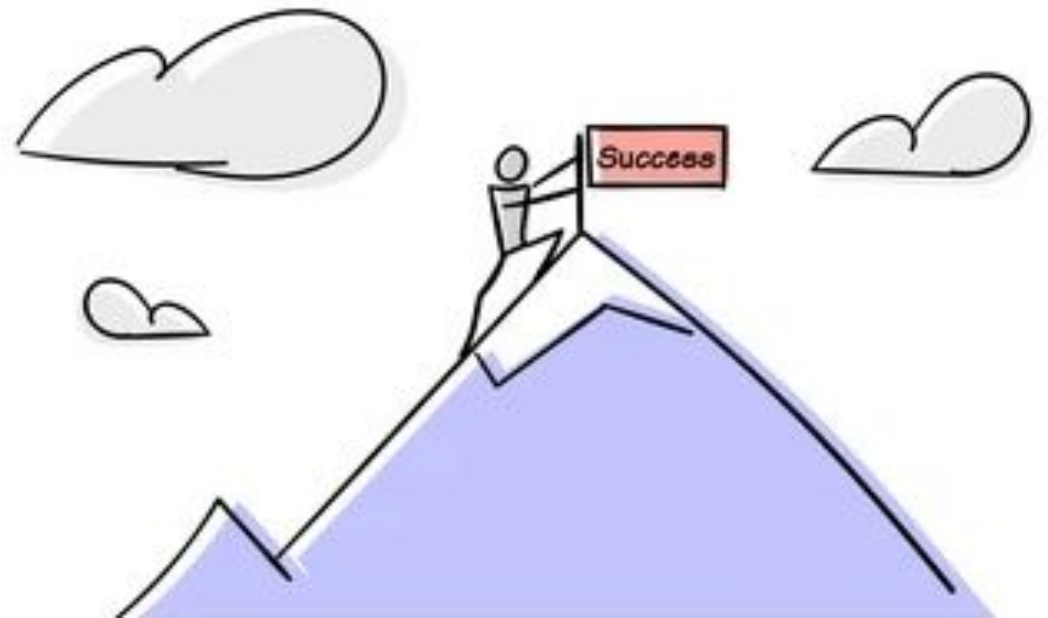


Hill Climbing Search

□ Hill-climbing search

- Keeps track of one current state.
- Moves to the neighboring state with the highest value in each iteration.
- Terminates when it reaches a "peak" where no neighbor has a higher value.

- However, it does not look ahead beyond the immediate neighbors of the current state.



Hill Climbing Search

- A **hill-climbing** algorithm (steepest-ascent version) is shown below.

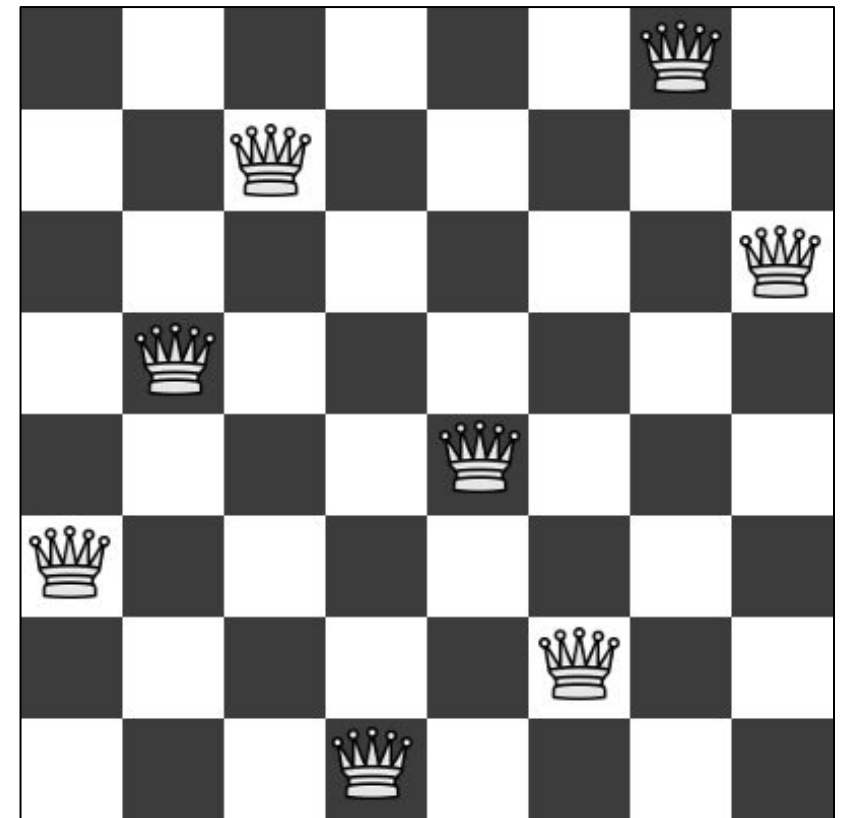
```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current  $\leftarrow$  problem.INITIAL
  while true do
    neighbor  $\leftarrow$  a highest-valued successor state of current
    if VALUE(neighbor)  $\leq$  VALUE(current) then return current
    current  $\leftarrow$  neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor.

-
- This algorithm has a node that comprises two parts: **state** and **value**.

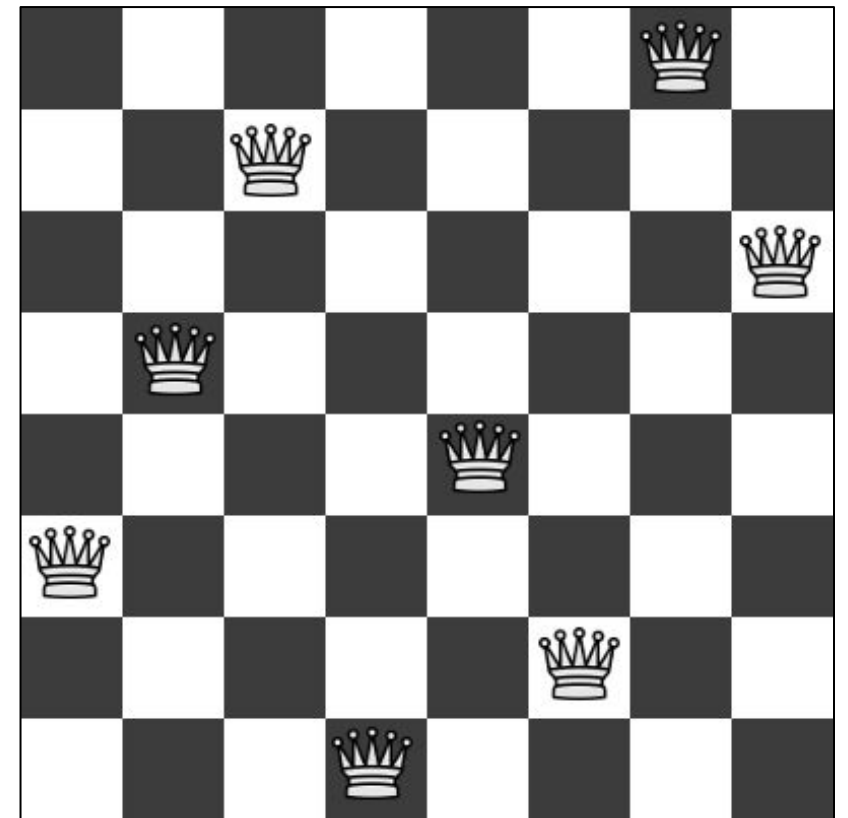
The 8-Queens Problem

- Local search algorithms typically use a **complete-state formulation**.
- Each state has 8 queens on the board, with one per column.
- The successors of a state are all possible states generated by moving a single queen to another square **in the same column**.
- **Question:** How many successors for each state?
 - $8 \times 7 = 56$ successors



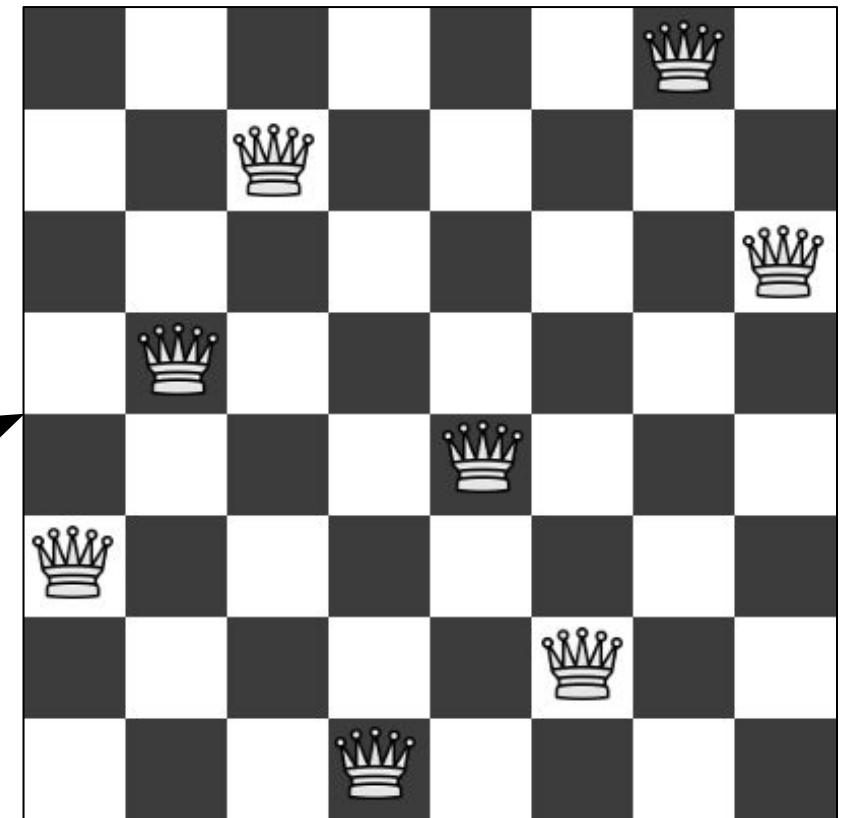
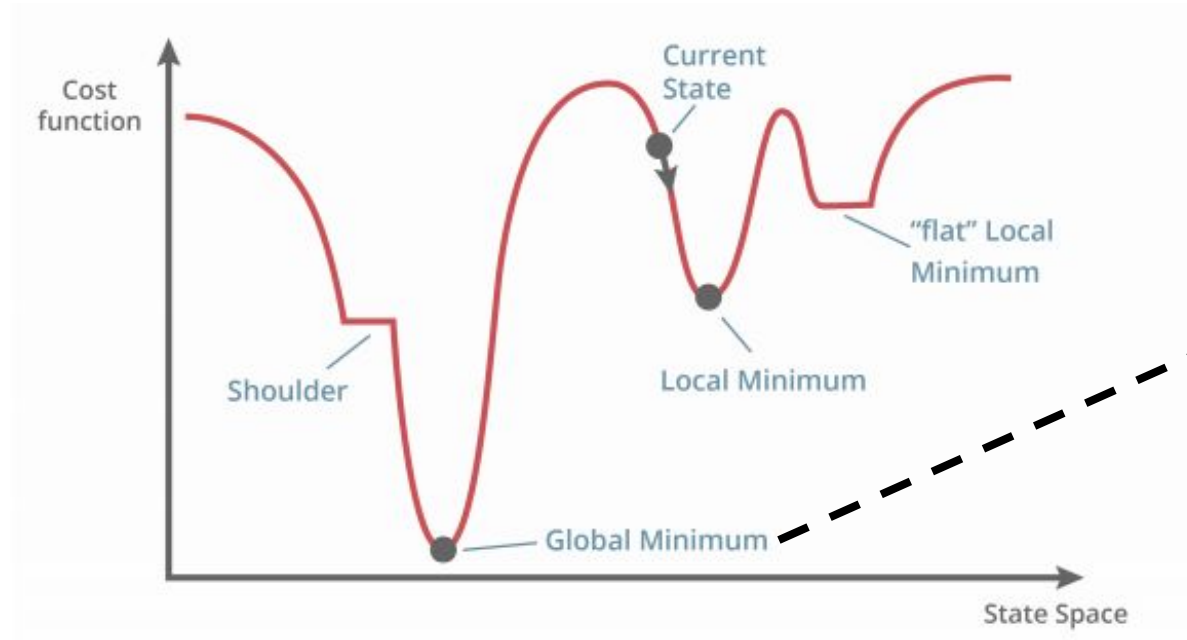
The 8-Queens Problem

- The heuristic cost function h is **the number of pairs of queens that are attacking each other**, either directly or indirectly.



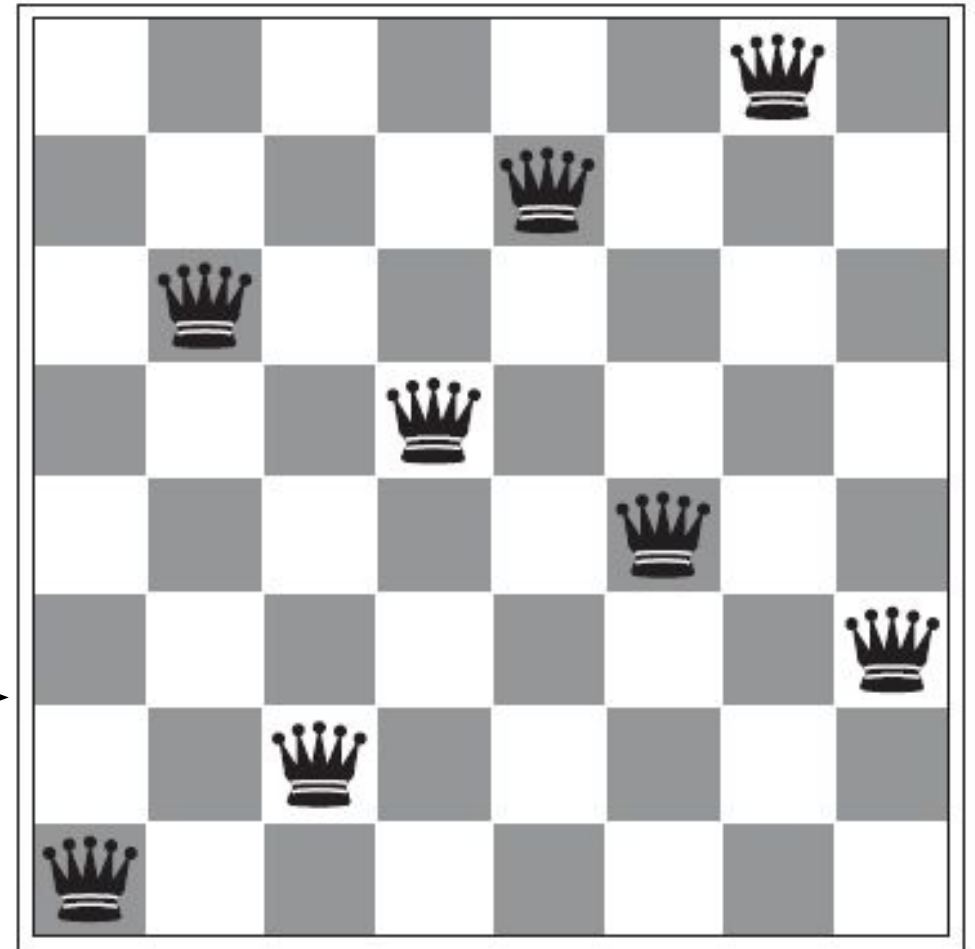
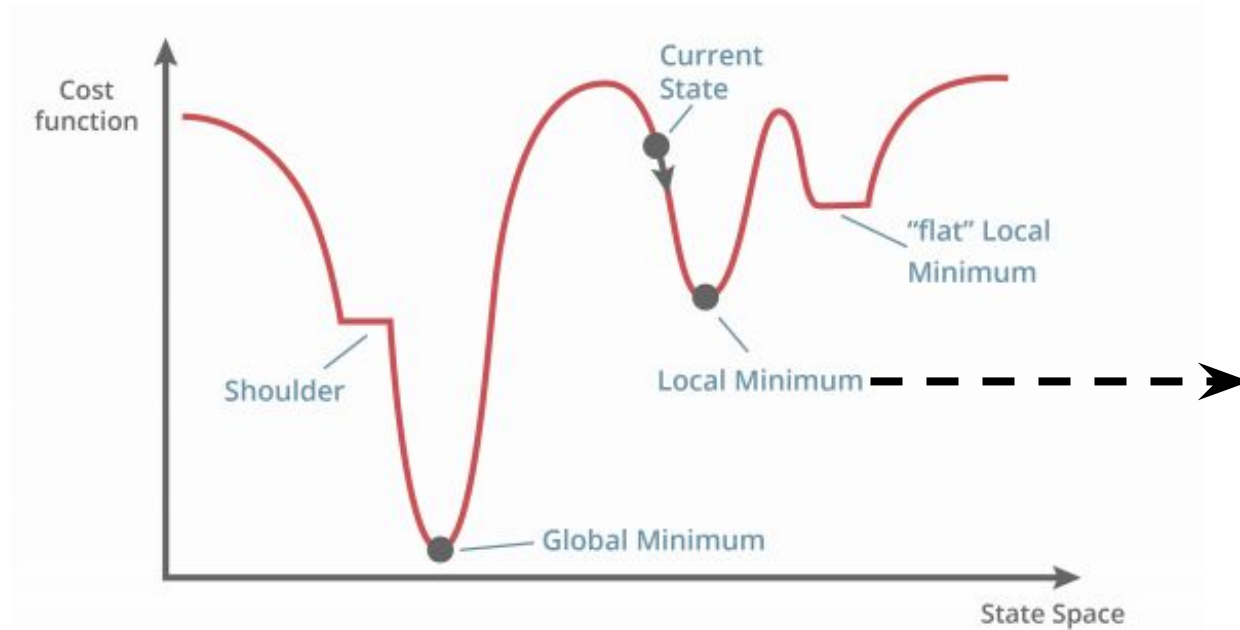
The 8-Queens Problem

- The **global minimum** of this cost function is zero, which occurs only at the best solution.



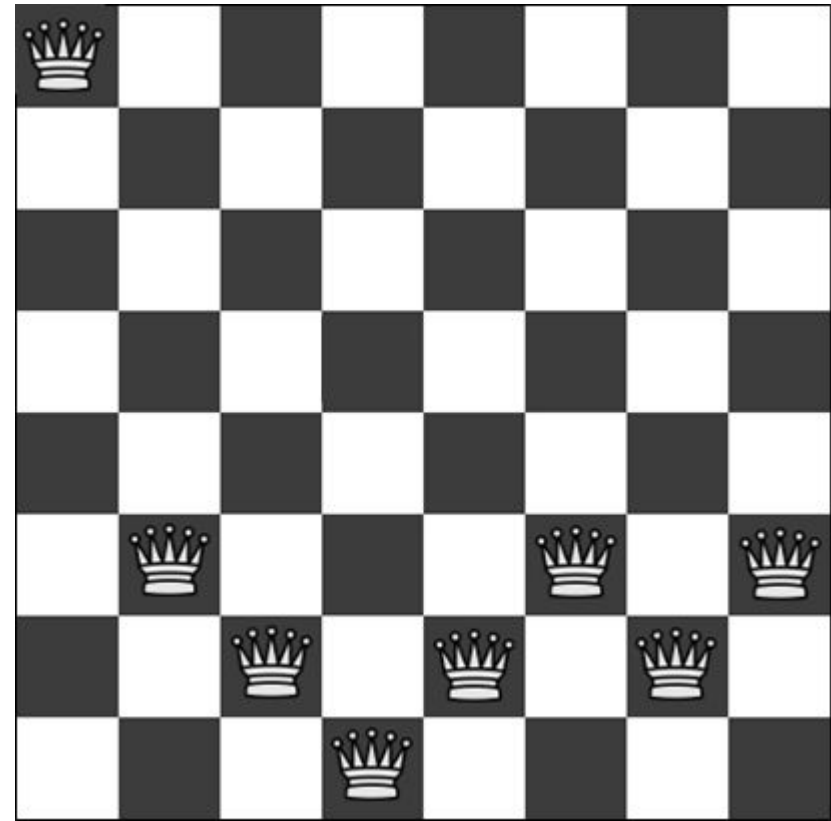
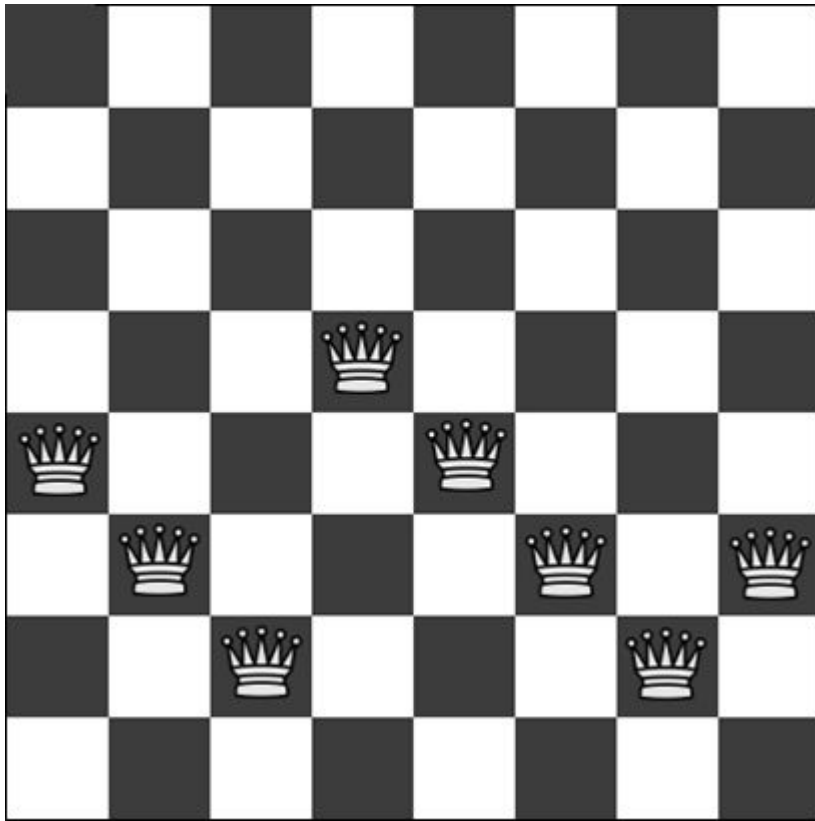
The 8-Queens Problem

- A **local minimum** in the 8-queens state space with $h = 1$ but every successor has a higher cost.



The 8-Queens Problem

□ **Question:** What are the h values of these two states?



The 8-Queens Problem

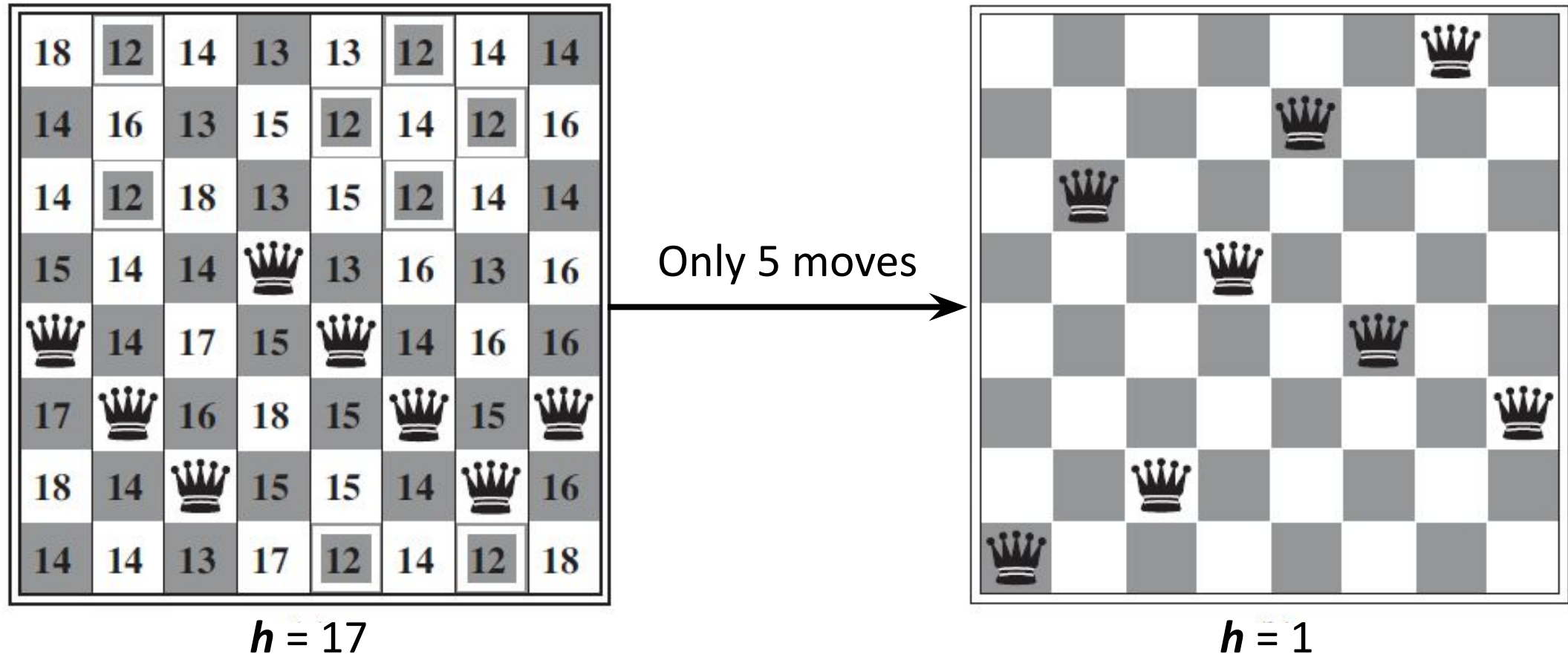
- An 8-queens state with a heuristic cost estimate of $h = 17$.
- The h -values for each possible successor obtained by moving a queen within its column are also displayed.
- The best successors have $h = 12$.
- **Question:** Which successor to choose?

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

Hill Climbing Search: Pros

- A **greedy** approach
 - It is sometimes called the greedy local search.
 - Only consider good neighboring state without thinking ahead.
- A **quick** approach with **acceptable performance**
 - Can often make rapid progress toward a solution.
 - An efficient solution to improve a bad state.
 - Requires low memory consumption.
 - Requires low computational power.

Hill Climbing Search: Pros



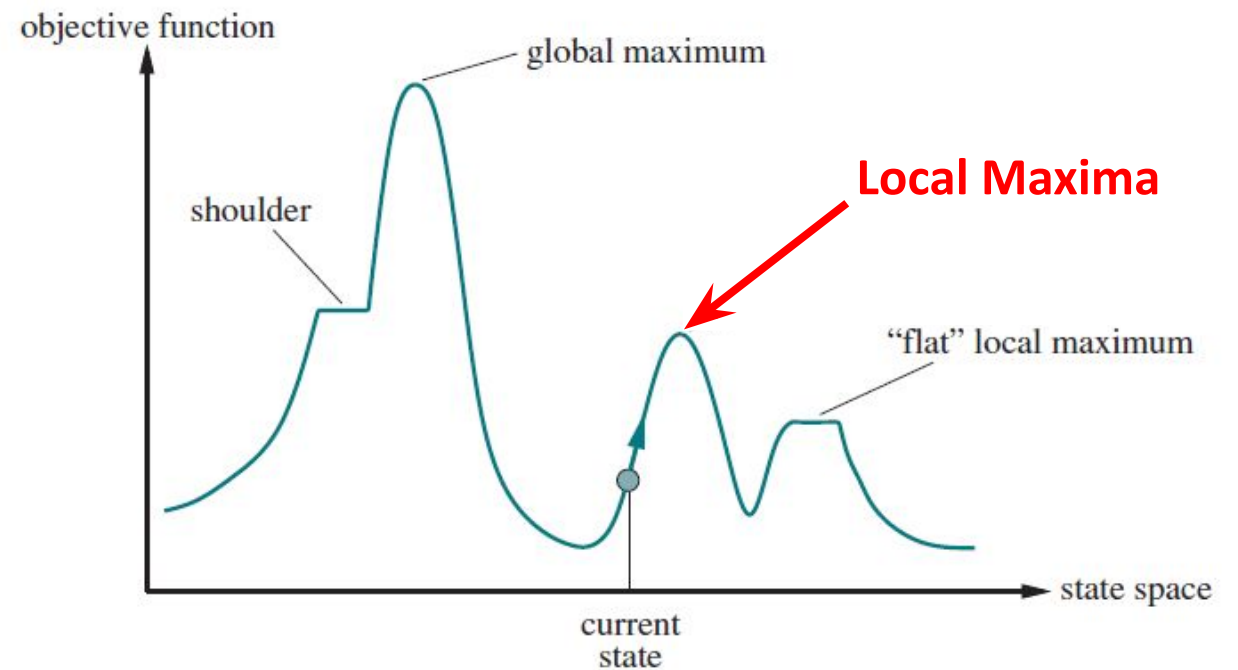
❑ **Question:** How to solve this problem?

Hill Climbing Search: Limitation

□ Hill climbing search often gets stuck at a **sub-optimal solution**.

□ Local maxima

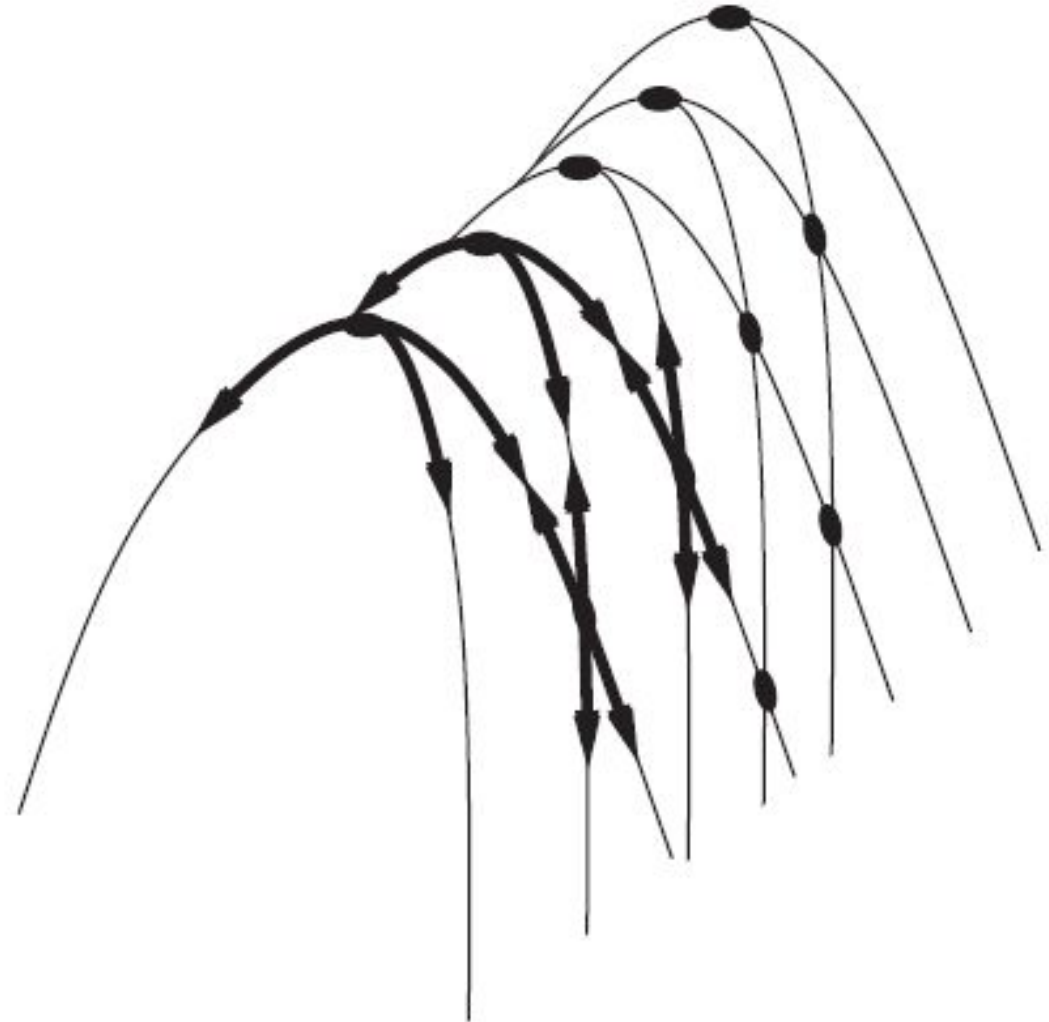
- A peak that is higher than each of its neighboring states but lower than the global maximum.
- Once arrived at the local maxima, it will be stuck with nowhere else to go.



Hill Climbing Search: Limitation

□ Ridges

- A sequence of local maxima that is difficult for greedy algorithms to navigate.
- From each local maximum, all the available actions point downhill.



Hill Climbing Search: Limitation

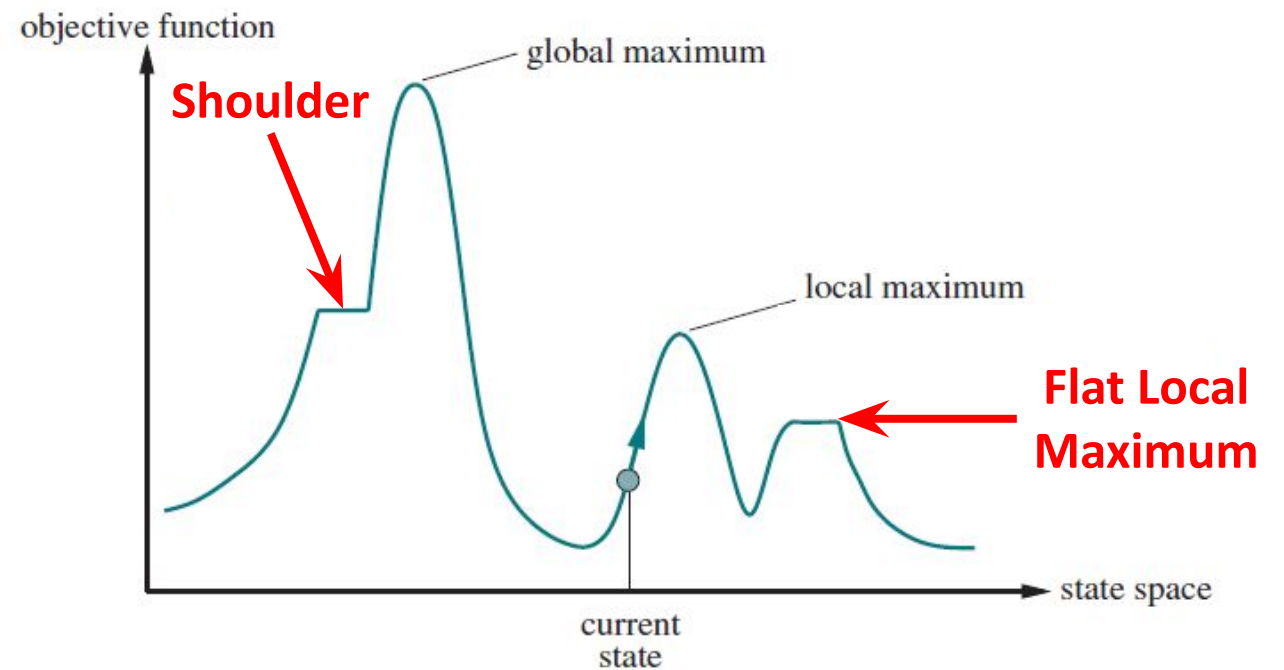
□ Plateaus

- A flat area of the state-space landscape.

 - A **flat local maximum**

 - A **shoulder**

- Search can get lost wandering on the plateau.



Hill Climbing Search: Limitation

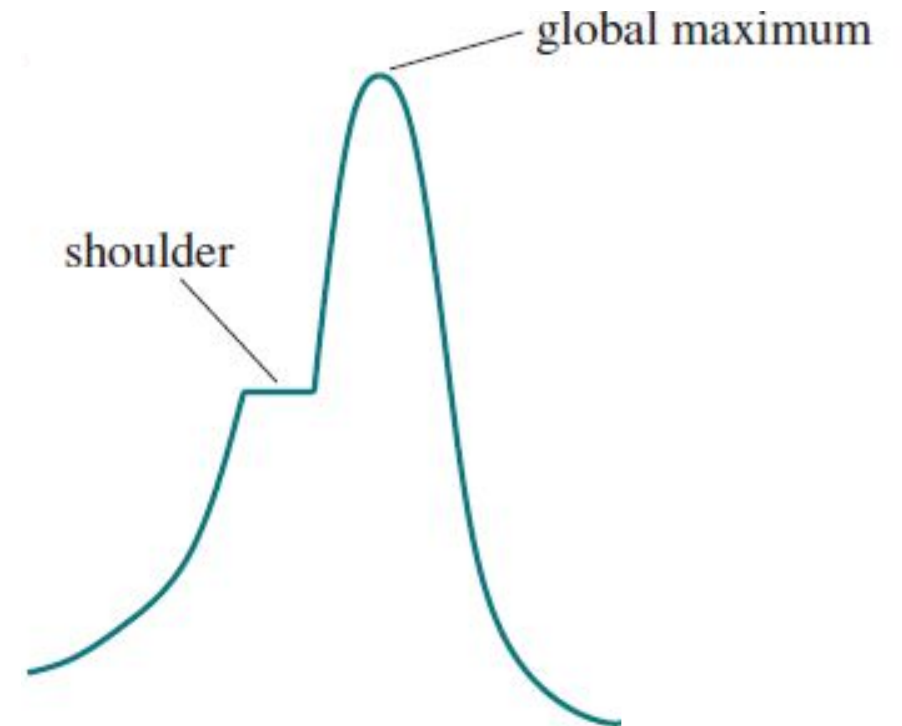
□ The 8-Queens problem

- State space: $8^8 \approx 17$ million states.
- Initial state: A randomly generated 8-queens state.
- Resulting state
 - Stuck (86%): 3 steps in average.
 - Success (14%): 4 steps in average.

□ Question: Is hill climbing a good solution?

Hill Climbing Search: Limitation

- Possible solution
 - To allow a **sideways move** in the hope that the plateau is really a **shoulder**.
 - Is this a rational move?
- Alternative solution
 - Limit: 100 consecutive sideways moves in the 8-queens problem.
 - Resulting state
 - Stuck (6%): 64 steps in average.
 - Success (94%): 21 steps in average.



Hill Climbing Search: Limitation

- Alternative solution
 - Utilize the **backtracking** technique.
 - Maintain a list of promising visited states.
 - If the search reaches an undesirable state, it can backtrack to the previous configuration and explore a new path.

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	👑	13	16	13	16
👑	14	17	15	👑	14	16	16
17	👑	16	18	15	👑	15	👑
18	14	👑	15	15	14	👑	16
14	14	13	17	12	14	12	18

Variants of Hill Climbing Search

□ Stochastic hill climbing

- Chooses at **random** from among the uphill moves.
- The probability of selection can vary with the steepness of the uphill move.

□ First-choice hill climbing

- A variant of the **stochastic hill climbing**.
- Chooses the first better state from **randomly** generated neighbors.
- This is a good strategy when a state has many successors.

Variants of Hill Climbing Search

□ Random-restart hill climbing

- Conducts a series of hill-climbing search from **randomly** generated initial states until a goal is found.
- After many rounds, the global maxima will eventually be found.
- Unfortunately, if there is no local maxima, then it will always get the same best state.

Hill Climbing Search: Insights

- A hill-climbing algorithm is generally **efficient**, but its nature of never making "downhill" moves is guaranteed to be **incomplete**.
 - Hill climbing search
- A purely random walk that moves to a successor chosen uniformly at random from the set of successors is **complete** but extremely **inefficient**.
 - Stochastic hill climbing
 - First-choice hill climbing
 - Random-restart hill climbing

Simulated Annealing

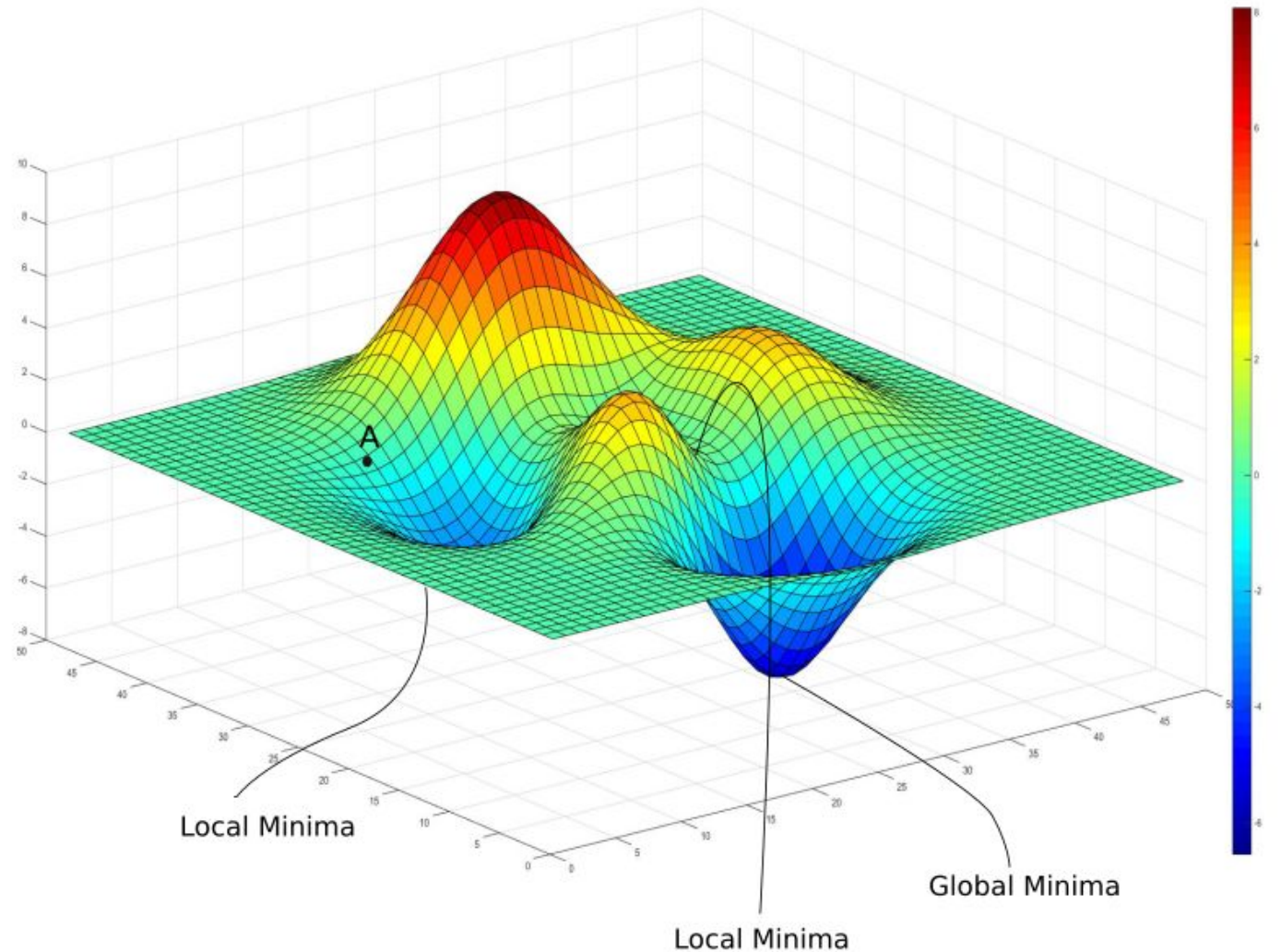
□ Annealing

- It is a technique, where a metal is heated to a high temperature to reduce its hardness and increase its ductility, before slowly cooling it down to improve its physical properties.
- When the metal is hot, the molecules randomly re-arrange themselves at a rapid pace.

- **Simulated annealing** is an algorithm that mimics the process of **annealing** by making the search going uphill and downhill that yields both **efficiency** and **completeness**.

Simulated Annealing

- A cost function is used to represent the Y-axis.
- **Gradient descent** is an iterative optimization algorithm to find the minimum of a function.
- Now, the aim is to find the global minimum.



Simulated Annealing

- Imagine the task of getting a ping-pong ball into the deepest crevice in a bumpy surface. If we just let the ball roll, it will come to rest at a local minimum.
- If we shake the surface, we can bounce the ball out of the local minimum.
- The trick is to shake just hard enough to bounce the ball out of local minima but not hard enough to dislodge it from the global minimum.
- The simulated-annealing solution is to:
 - Start by shaking hard (i.e., at a high temperature, T).
 - Then, gradually reduce the intensity of the shaking (i.e., lower T).

Simulated Annealing Algorithm

- Instead of picking the best move like how hill climbing does, simulated annealing algorithm picks a **random move**.

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current  $\leftarrow$  problem.INITIAL
  for  $t = 1$  to  $\infty$  do
     $T \leftarrow$  schedule( $t$ )
    if  $T = 0$  then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  VALUE(current) – VALUE(next)
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 
```

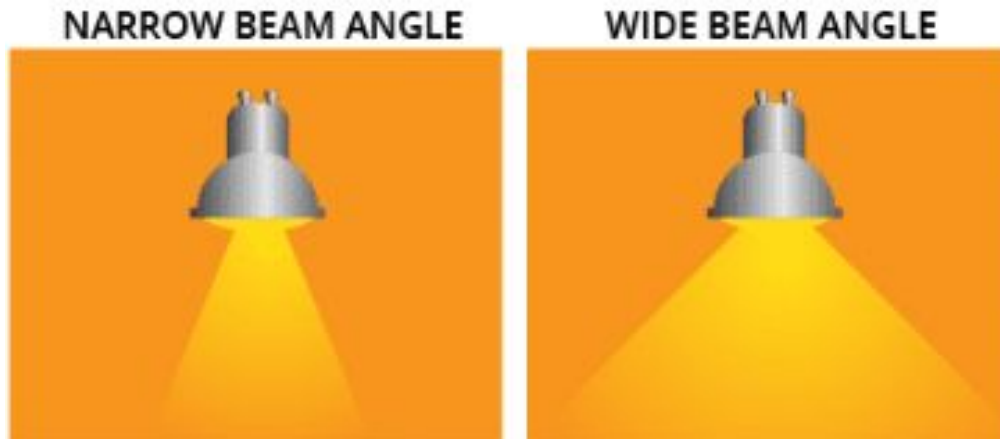
Figure 4.5 The simulated annealing algorithm.

Simulated Annealing

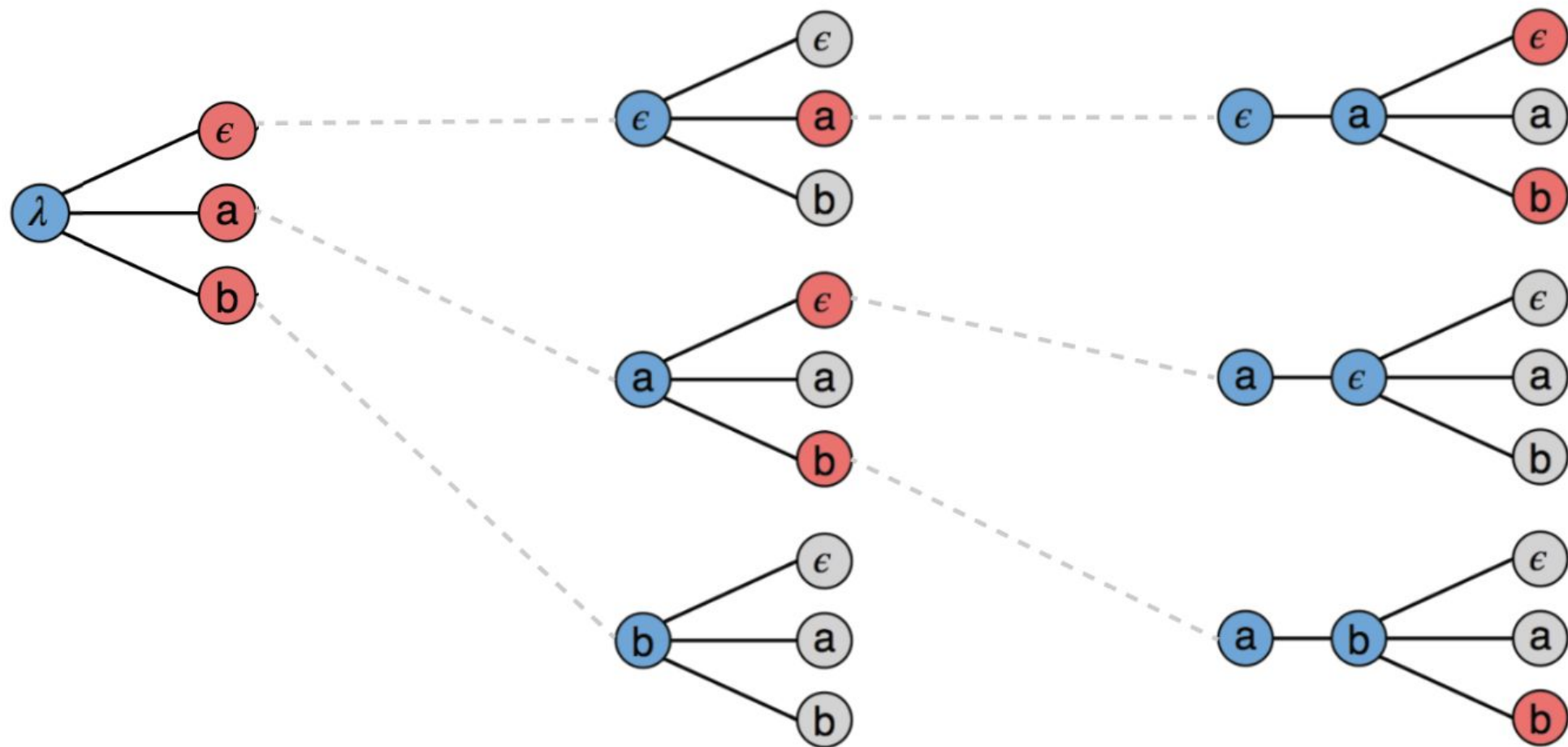
- Simulated annealing can be considered as an **upgrade version** of stochastic hill-climbing.
 - **Stochastic Hill Climbing**
 - A move is made if the new state is an improvement to the current state.
 - **Simulated Annealing**
 - Similar to stochastic hill climbing.
 - However, sometimes worse states are also accepted to allow the algorithm to learn answers that are eventually better.

Local Beam Search

- The **algorithm** keeps track of k states (or **beam width**) rather than just one.
- It begins with k randomly generated states.
- At each step, all the successors of all k states are generated.
- If any one is a goal, the algorithm halts.
- Otherwise, it selects the k best successors from the complete list and repeats.



Local Beam Search



Local Beam Search

- **Random-restart hill climbing search**

- Each search process runs independently of the others.

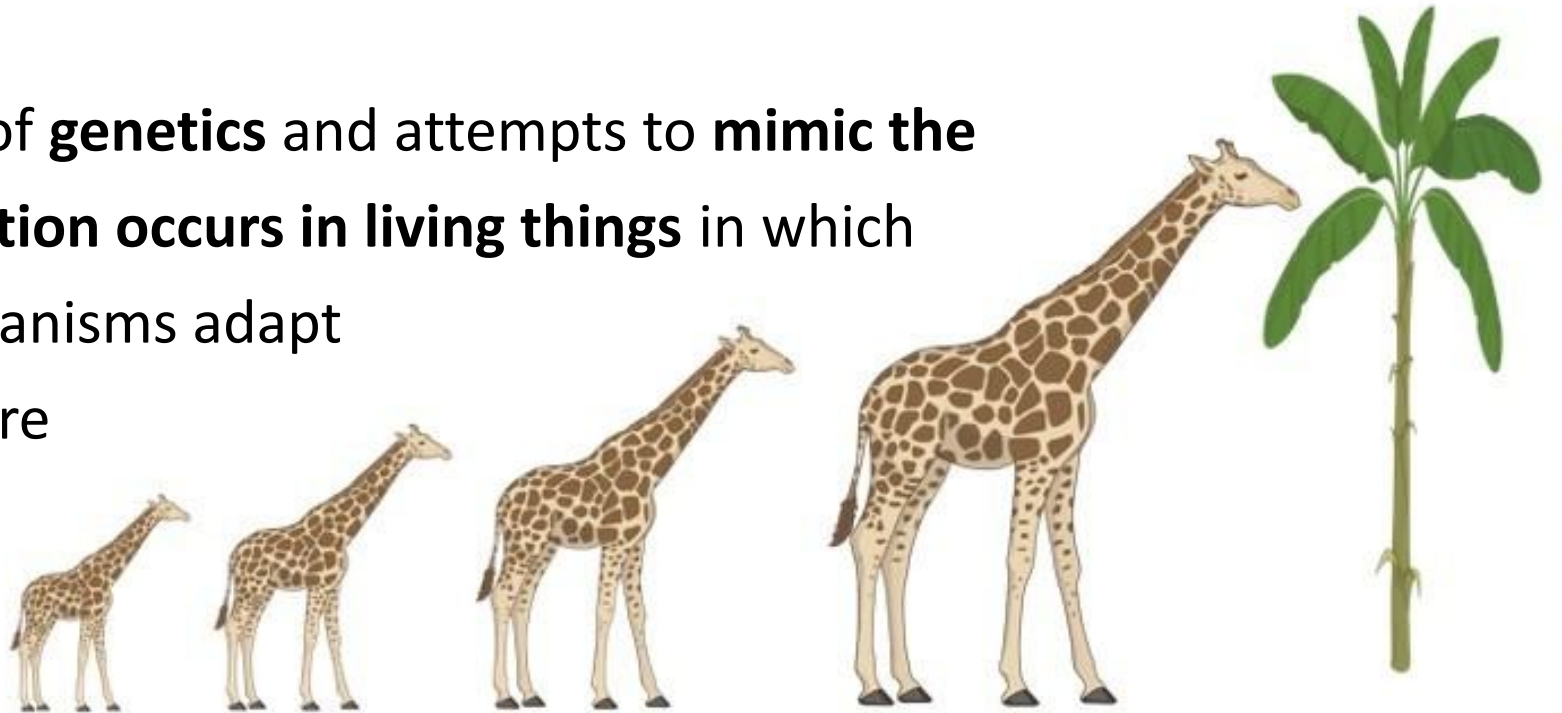
- **Local beam search**

- Useful information is passed among the parallel search threads.
 - Unfruitful searches are abandoned immediately.
 - The search moves its resources to where the most progress is being made.
 - As a consequence, search tends to get stuck at local maxima or plateau.

- A solution to this problem is a variant called **stochastic beam search**.

Evolutionary Algorithms

- A **evolutionary algorithm** can be seen as a variant of stochastic beam search in which successor states are generated by combining two parent states rather than by modifying a single state.
- It employs the concept of **genetics** and attempts to **mimic the process of natural selection occurs in living things** in which populations of living organisms adapt to change so that they are more likely to survive.



Evolutionary Algorithms

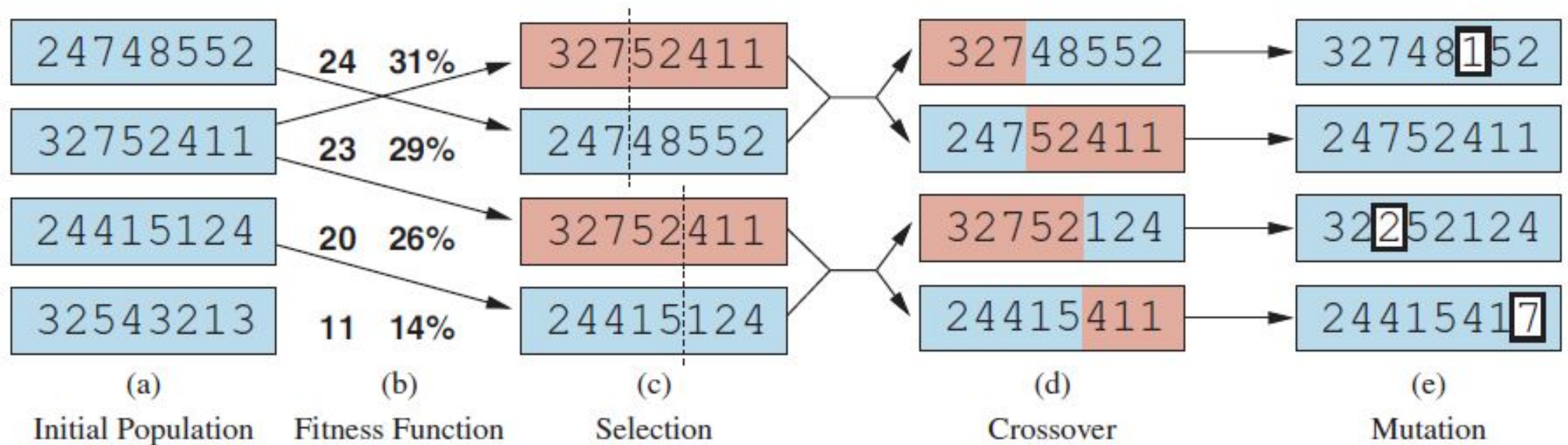
- Evolutionary algorithms can search through different combinations to identify the perfect combination.
 - Example of optimization problems
 - The 8-queens problem
 - University course scheduling problem
 - Cutting stock problem
 - Traveling-salesman problem
 - etc.

Evolutionary Algorithms

- In general, evolutionary algorithms follow the following 5 phases to solve complex optimization problems:
 - Initialization
 - Fitness Function
 - Selection
 - Crossover
 - Mutation

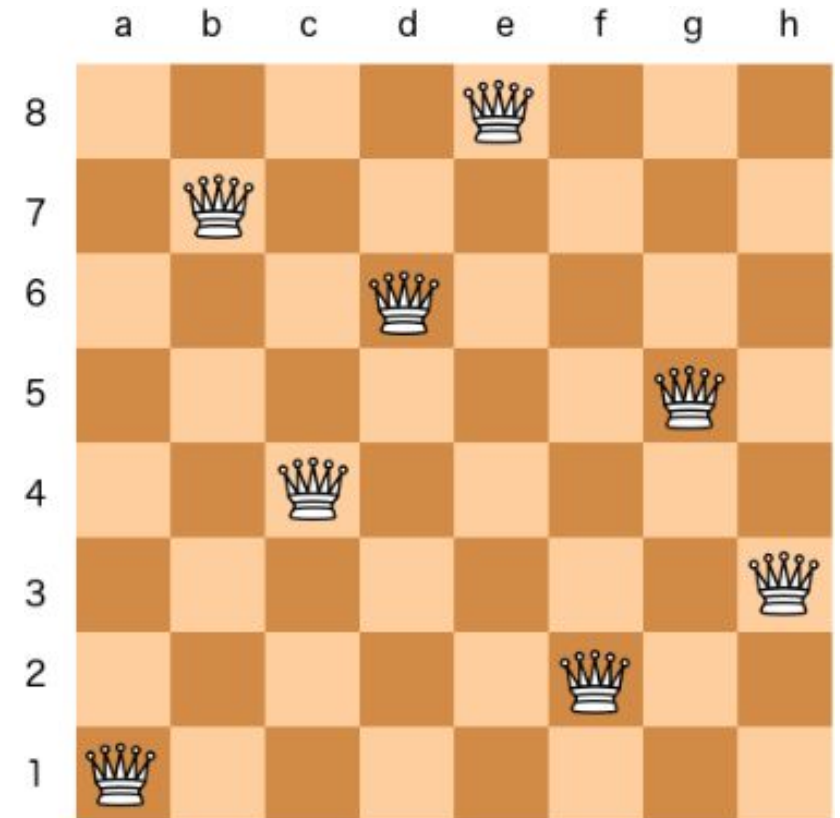
Evolutionary Algorithms: 8-Queens Problem

- Figure below shows the production of the next generation via the 5 phases of an evolutionary algorithm.



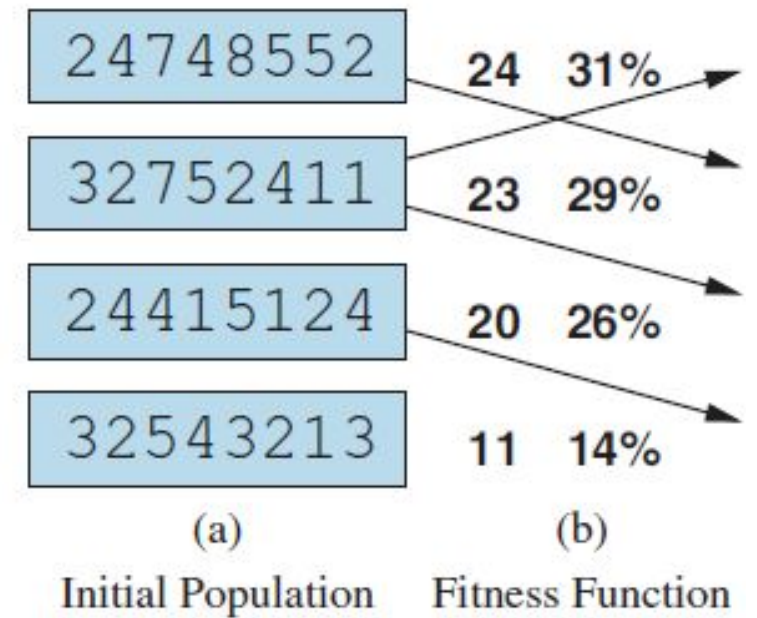
Evolutionary Algorithms: Initialization

- Evolutionary algorithms begin with a set of k randomly generated states, called the **population**.
- Each state is represented as an 8 digit string.
- E.g., 17468253.
- The position of each digit represents a column.
- The value of each digit represents the row number of a column.



Evolutionary Algorithms: Fitness Function

- Each state is usually rated by an **objective function**, or a **fitness function**.
- For the 8-queens problem, the **number of non-attacking pairs of queens** is used as the heuristic.
 - The scores of the 4 states are 24, 23, 20, and 11.
 - A solution has a score of 28.
- The scores are then normalized to probabilities, in which a **better state** has a **higher chance to be selected**.



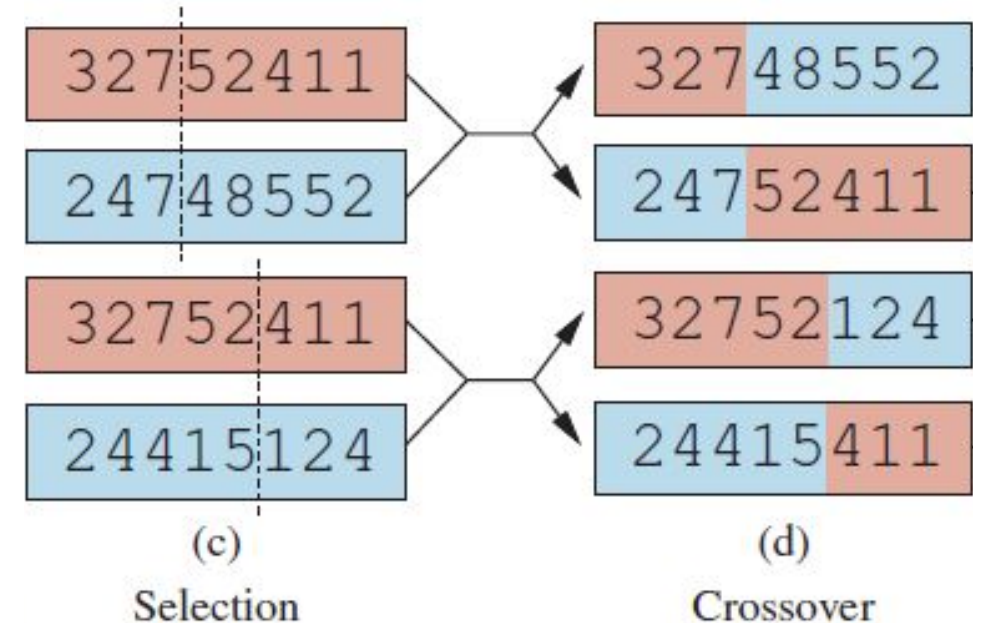
Evolutionary Algorithms: Selection

- Two pairs are selected at random for reproduction, in accordance with the **probabilities** generated in step (b).
- For each selected pair to be mated, a **crossover point** is chosen randomly.
- In this case, the crossover points are after the third digit in the first pair and after the fifth digit in the second pair.



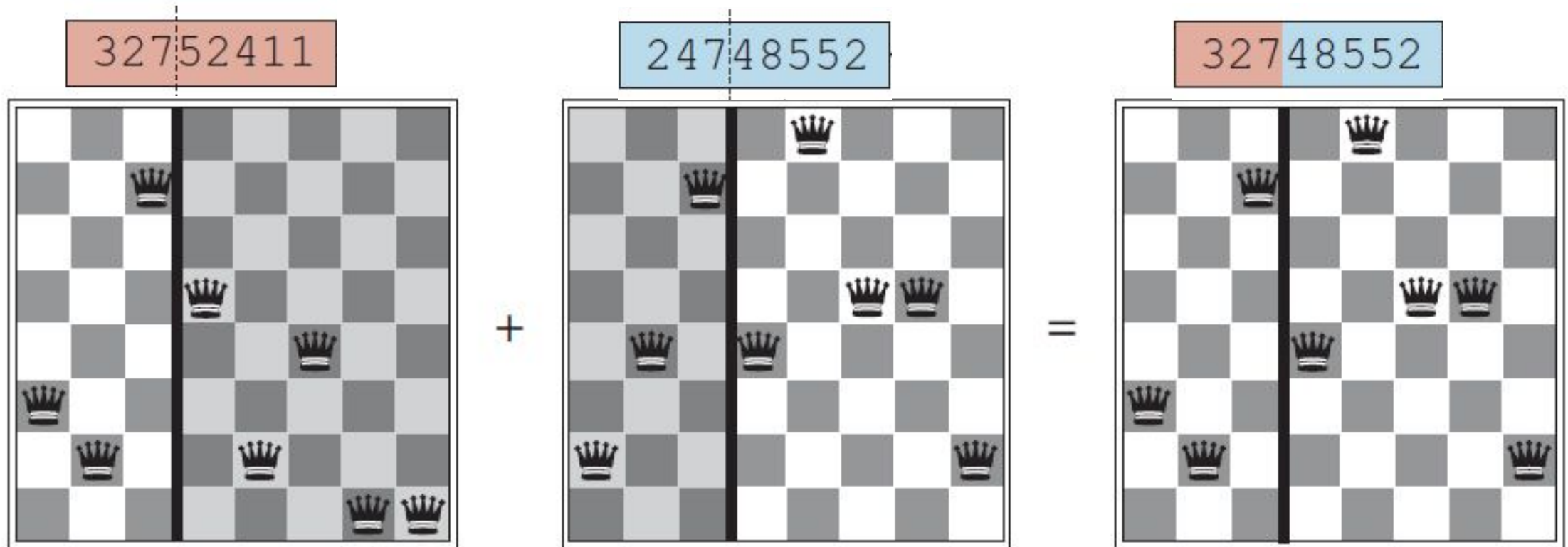
Evolutionary Algorithms: Crossover

- In step (d), the offspring themselves are created by crossing over the parent strings at the crossover point.
- E.g., the first child of the first pair gets the first 3-digit block (e.g., 327) from the first parent and the remaining 5-digit block from the second parent (e.g., 48552).



Evolutionary Algorithms: Crossover

- The 8-queens states involved in this step are shown below.



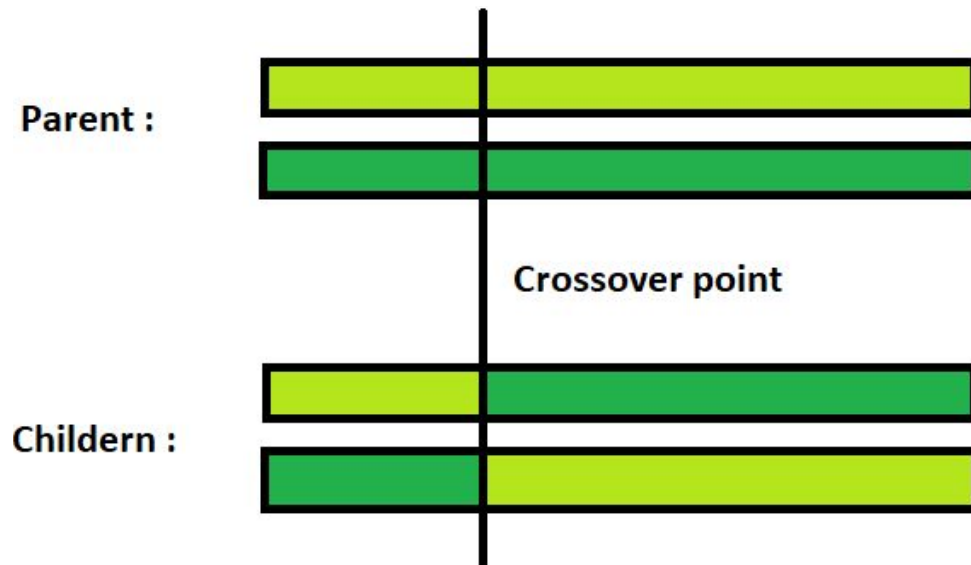
Evolutionary Algorithms: Crossover

- When two parent states are quite different, the crossover operation can produce a state that is a long way from either parent state.
- It is often the case that the population is quite diverse early on in the process.
- Crossover often takes large steps in the state space early in the search process.
- It takes smaller steps later on when most individuals are quite similar.
- When this happens, a genetic algorithm is said to **converge**.
- **Convergence** is a phenomenon in evolutionary computation that causes evolution to halt because precisely every individual in the population is identical.

Evolutionary Algorithms: Crossover

□ Single Point Crossover

- A crossover point on the parent string is selected.
- All data beyond that point in each of the parent string is swapped between the two parents.

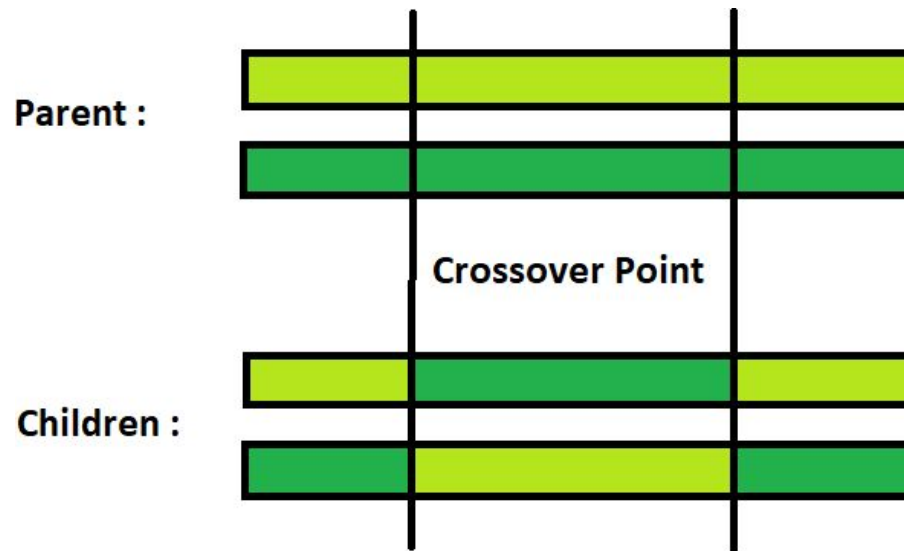


Chromosome1	11011 00100110110
Chromosome2	11011 11000011110
Offspring1	11011 11000011110
Offspring2	11011 00100110110

Evolutionary Algorithms: Crossover

□ Two-Point Crossover

- This is a specific case of a N-point Crossover technique.
- Two random points are chosen on the individual strings and the genetic material is exchanged at these points.



Chromosome1	11011 00100 110110
Chromosome2	10101 11000 011110
Offspring1	11011 11000 110110
Offspring2	10101 00100 011110

Evolutionary Algorithms: Crossover

- Uniform Crossover
 - Each bit is selected randomly from one of the corresponding bits of the parent.
 - Use tossing of a coin as an example technique.

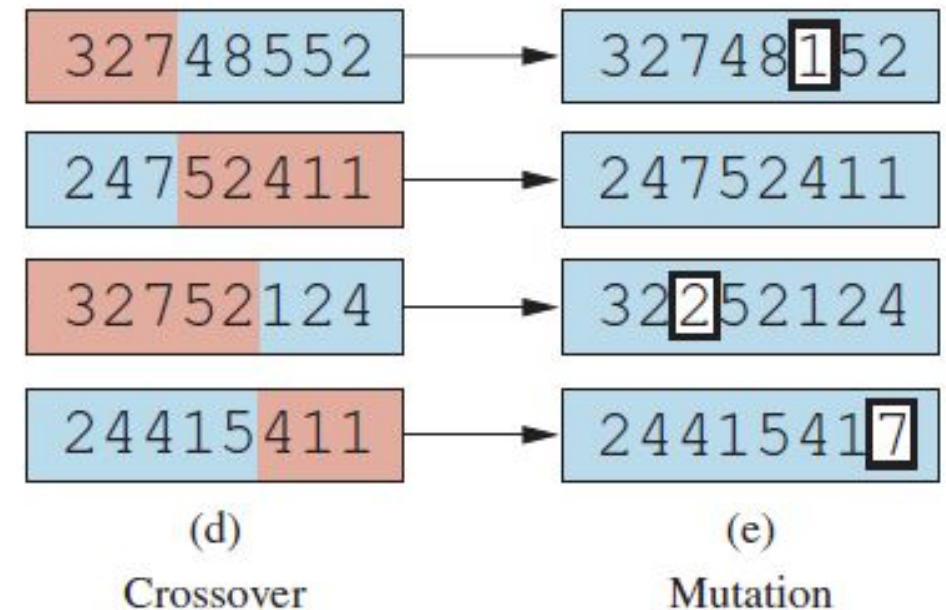


Evolutionary Algorithms: Crossover

- The **primary advantage** of evolutionary algorithms comes from the **crossover** operation, if the blocks perform **useful functions**.
- For example, the first 3 queens in positions 2, 4, and 7 constitutes a **useful block** which can construct a solution when combined with other blocks.
- Blocks can implemented with **schema**, a substring with some of the positions left unspecified.
- For example, the schema 247***** describes all 8-queens states in which the first 3 queens are in positions 2, 4, and 7, respectively.
- Strings that match the schema (e.g., 24713578) are called **instances** of the schema.

Evolutionary Algorithms: Mutation

- Each location is subject to random mutation with a small independent probability.
- One digit was mutated in the first, third, and fourth offspring.
- In the 8-queens problem, this corresponds to choosing a queen at random and moving it to a random square in its column.



Evolutionary Algorithms: Different Setups

- Evolutionary algorithms can exist in endless forms based on different setups.

- **Initialization**

- The **size** of the population.

- The **representation** of each individual.

- In **genetic algorithms**, each individual is a **string over a finite alphabet**.

- In **evolution strategies**, an individual is a **sequence of real numbers**.

- In **genetic programming**, an individual is a **computer program**.

Evolutionary Algorithms: Different Setups

□ Selection

- The mixing number (ρ) is the number of parents combined to form offspring:
 - All individuals with probability proportional to their fitness score.
 - ρ most fit candidates from randomly selected n individuals.

□ Crossover

- When $\rho = 1$, it becomes the stochastic beam search strategy.
- It is possible to have $\rho > 2$, but the most common case is $\rho = 2$ where two parents combine their "genes" to form offspring.

Evolutionary Algorithms: Different Setups

□ Mutation

- The **mutation rate** determines how often offspring have random mutations to their representation.
- Once an offspring has been generated, every bit in its composition is flipped with probability equal to the mutation rate.

□ Makeup of the New Population

- The newly formed offspring.
- It can also include a few top-scoring parents from the previous generation to guarantee that overall fitness will never decrease over time.

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Figure 4.8 A genetic algorithm.

Evolutionary Algorithms: Termination

- **Question:** When will the reproduction cycle terminates?
- The termination conditions could be:
 - When there is no improvement in the population.
 - When a predefined number of the reproduction cycle has already been fulfilled.
 - When the fitness function has reached a predefined value.

Evolutionary Algorithms: Insights

- Evolutionary algorithms combine the following strengths of other strategies:
 - An uphill tendency.
 - Random exploration.
 - Exchange of information among parallel search threads.

- **Question:** Which few search strategies?

- **Question:** Which phase of evolutionary algorithms correspond to the above strategies?

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 4: Search in Complex Environments (page 128 to page 137)

The End